Operating Systems

Dmitry Zaitsev

Lecture 22:
File system of OS. Case study: file structure of disc in Linux.

Control of information: file system of OS

- Information as a resource of OS: structured storage for files
- File – a unit to store information – named piece of information of a certain type
- Basic file types: textual and binary
- Sometimes type of file is distinguished by the name extension: .jpg, .html, .docx
- Files are assembled into directories (folder) base on the recurrent principle: a directory might content directories and files – nested system of directories
- Volume represents a replaceable hardware unit to store information – mount and unmount a volume

File System Hierarchy

Volumes contain directories, which can contain subdirectories and files.

VOLUMES

A volume is a storage device or partition.

DIRECTORIES

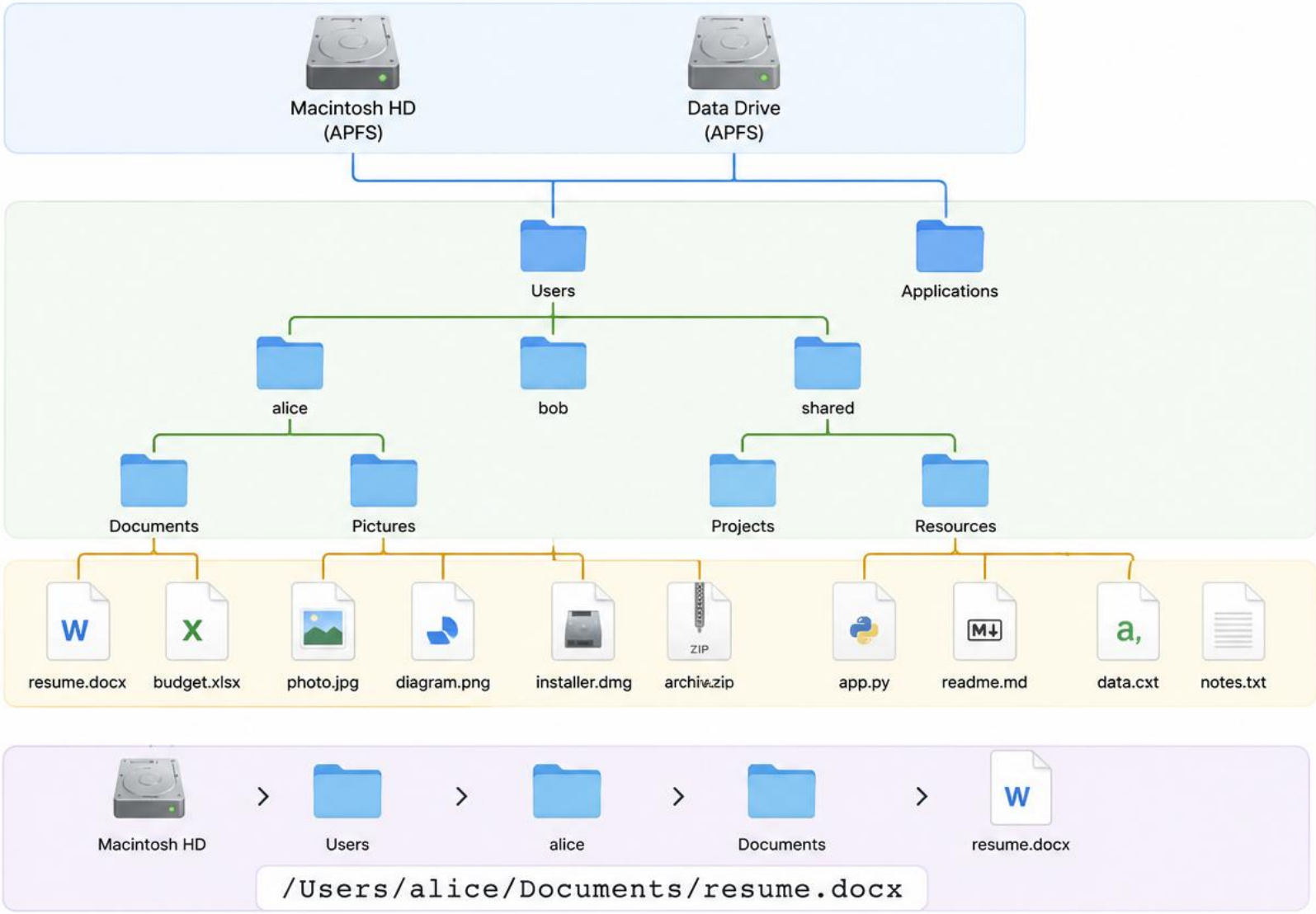
Directories (folders) can contain subdirectories and files.

FILES

Files are the actual data stored in directories.

EXAMPLE PATH

A file's location is defined by its path.



Files (documents)

- Sequential or direct access
- Basic operations: Open, Close, Read, Write, Rewind
- Additional operations for direct access: Seek – positioning within file
- Other types of files are implemented by databases
- Some OS, provided more sophisticated files like index-sequential in OS 360
- Textual and binary I/O operations
- For binary files – number of bytes
- For textual files – some formatted units, strings and numbers

Directory record

- name
- record type – file or directory
- file type – textual of binary
- owner information
- protection (access) information
- dates – creation, modification, access
- length – number of bytes
- mapping on media (first cluster for FAT)
- other info

Directory Record in an Operating System

A directory is a special file that contains a list of records. Each record describes a file or a subdirectory.

Example: Directory Record (for a file named “report.txt”)

File Name	Inode Number	File Type	Permissions (Mode)	Link Count	Owner User ID	Group ID	File Size (bytes)	Timestamps			Flags
								Last Access	Last Modified	Last Change	
report.txt	1048576	Regular File	-rw-r--r-- (0644)	1	1000	1000	12,458	2025-05-20 10:15:30	2025-05-20 09:58:12	2025-05-20 09:58:12	0x00

- **File Name** – Name of the file or subdirectory (variable length)
- **Inode Number** – Unique identifier of the file’s metadata (index node)
- **File Type** – Type of the entry: regular file, directory, symbolic link, device, etc.
- **Permissions (Mode)** – Access rights for owner, group, and others (e.g., rwxr-xr-- or 0644)
- **Link Count** – Number of directory entries (hard links) referring to this file
- **Owner User ID** – User ID of the file owner
- **Group ID** – Group ID of the file
- **File Size** – Size of the file in bytes (for directories, size may be implementation-dependent)
- **Timestamps** – Important time information
  - **Last Access:** last time the file was read
  - **Last Modified:** last time the file content was modified
  - **Last Change:** last time the file’s metadata (e.g., permissions, links) was changed
- **Flags** – File attributes and status flags (e.g., immutable, hidden, compressed)

Common File Types

-	Regular File	(e.g., report.txt)
d	Directory	(e.g., **docs**)
l	Symbolic Link	(e.g., link.lnk)
c	Character Device	(e.g., /dev/tty)
b	Block Device	(e.g., /dev/sda)
p	Named Pipe	(FIFO)
s	Socket	(IPC)

Directory records map names to file metadata. The actual **file data is stored elsewhere** on disk; the record contains attributes needed to locate, access, and manage the file.

Note: Different file systems (e.g., ext4, NTFS, XFS) may store additional fields or use different formats, but the core attributes are similar.

Volume information

- Volume Identifier: The human-readable name (label) of the volume.
- Standard Identifier: A unique string (e.g., "CD001" for ISO 9660) that identifies the file system type.
- Version Number: The version of the volume descriptor format being used.
- Geometry and Layout: Volume Space Size, Logical Block Size, Volume Set Information
- Structural Pointers: Root Directory Record, Allocation Table
- Metadata and Ownership: Timestamps, Copyright/Abstract Files

OS control blocks for work with file system

- File Control Block (FCB) stores metadata about a file: file name (or reference to it); file size; file type; location on disk (pointers to disk blocks); permissions (read/write/execute); timestamps (creation, modification, access)
- Open File Table represents currently opened files in the system, contains: pointer to FCBFile access mode (read/write/append); current file offset (read/write position); open count (number of processes using it)
- File Descriptor Table (per process) represents a process’s view of open files, contains: file descriptors (integer handles like 0, 1, 2, 3…); pointers to entries in the Open File Table

MS-DOS file structure of disc

- **FAT file system**
- Disk divided into:
  - ✓Boot sector
  - ✓FAT tables
  - ✓Root directory
  - ✓Data area
- Files stored in clusters
- FAT maintains chain of clusters
- Root directory is fixed size

File Allocation Table (FAT)

- Directory record of files provides a pointer to the first cluster
- file1, length 5 clusters, first cluster 4
- file2, length 7 clusters, first cluster 2
- …
- Each record of FAT specifies a cluster and contains a link to the next cluster, starts from 2:

2: 0	3: 16	4: 5	5: 10	6: 0	7: 8	8: 21	9: 0	10: 11	11:12
12: EOF	13: 0	14: 3	15: 0	16: 17	17: 18	18: 7	19: 0	20: 0	21: EOF

Linux basic I/O functions

```
User program (streams)
        ↓
   FILE * (stdio buffer)
        ↓
   File descriptor (kernel level)
        ↓
   Kernel file system / device
```

- File descriptors are low-level kernel integers for I/O, while streams are high-level buffered abstractions built on top of file descriptors.
- Streams use buffered I/O: fopen() – open file stream; fclose() – close stream; fread() / fwrite() – binary I/O; fprintf() / fscanf() – formatted I/O
- A file descriptor is a low-level integer handle used by the kernel to track open files, devices, sockets, pipes, etc. Basic functions: open(), read(), write(), close()

File descriptor I/O example

```
int fd = open("file.txt", O_RDONLY);  // open file
char buf[100];
int n = read(fd, buf, 100);  // read from file
write(1, buf, n);  // write to file descriptor 1 (stdout)
close(fd);  // close file
```

Stream I/O example

```
FILE *fp = fopen("file.txt", "r"); // open file
char buf[100];
fgets(buf, 100, fp);   // read from file
fputs(buf, stdout);    // print to standard output
fclose(fp);
```

Linux commands to create disc file structure

- Identify the disk: lsblk; sudo fdisk –l
- Create a partition, historical: sudo fdisk /dev/sdb
- Create a partition, modern:

  sudo parted /dev/sdb
  mklabel gpt
  mkpart primary ext4 0% 100%
  quit

- Create filesystem (format partition)

  sudo mkfs.ext4 /dev/sdb1

- Create mount point (directory) and Mount the filesystem

  sudo mkdir -p /mnt/mydisk
  sudo mount /dev/sdb1 /mnt/mydisk

Volume tables: fstab, mtab

- What SHOULD be mounted at boot (static config): /etc/fstab
- What IS currently mounted (dynamic state): /etc/mtab
- cat /etc/mtab

  Example entry:

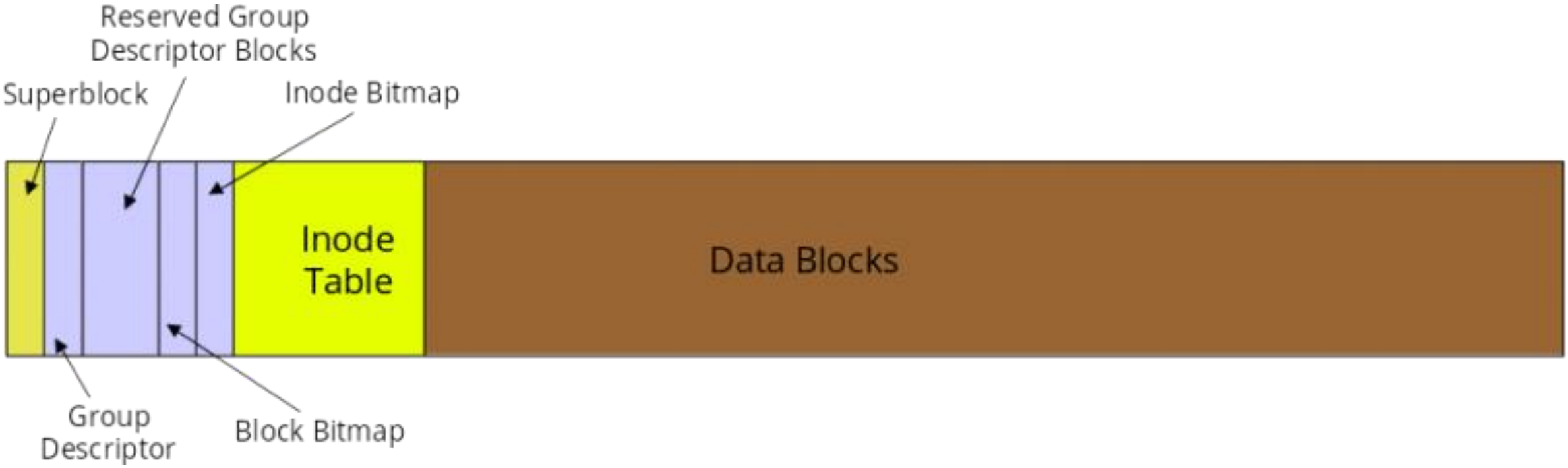
  /dev/sda1 / ext4 rw,relatime 0 0

  Which devices are mounted, where they are mounted (mount points), filesystem type, mount options
- mount / unmount
- findnmt
- df –h
- fsck -fn

Description of ext4

- ext4 (Fourth Extended Filesystem) is a high-performance journaling filesystem for Linux, supporting volumes up to 1 EiB and files up to 16 TiB, utilizing 48-bit block addressing. Key features include extents for improved large file handling, persistent preallocation, delayed allocation, fast filesystem checks (uninit_bg), and nanosecond timestamps
- ext4 allocates storage space in units of “blocks”. A block is a group of sectors between 1KiB and 64KiB, and the number of sectors must be an integral power of 2
- In ext4, data allocation was changed from fixed blocks to extents. An extent is described by its starting and ending place on the hard drive.

General structure of ext filesystems



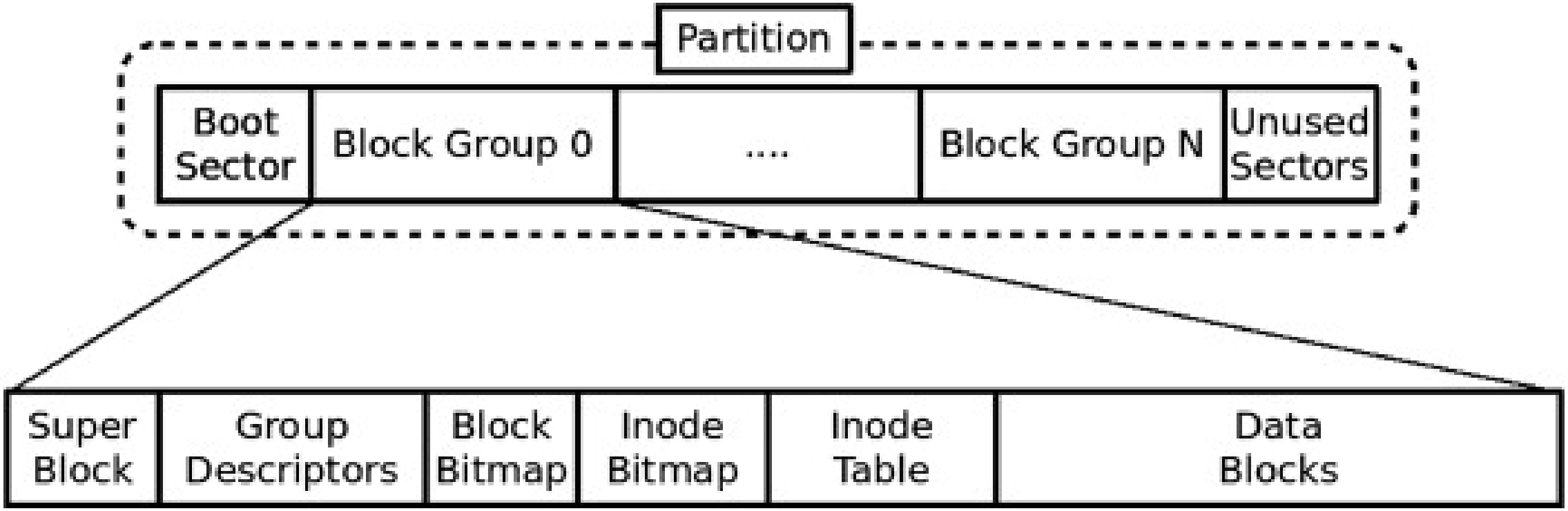
ext general structure description

- A **boot sector** in the first sector of the hard drive on which it is installed. The boot block includes a very small boot record and a partition table.
- The first block in each partition is a **superblock** that contains the metadata that defines the other filesystem structures and locates them on the physical disk assigned to the partition.
- An **inode bitmap block**, which determines which inodes are used and which are free.
- The **inodes**, which have their own space on the disk. Each inode contains information about one file, including the locations of the data blocks, i.e., zones belonging to the file.
- A **zone bitmap** to keep track of the used and free data zones.
- A **data zone**, in which the data is actually stored.

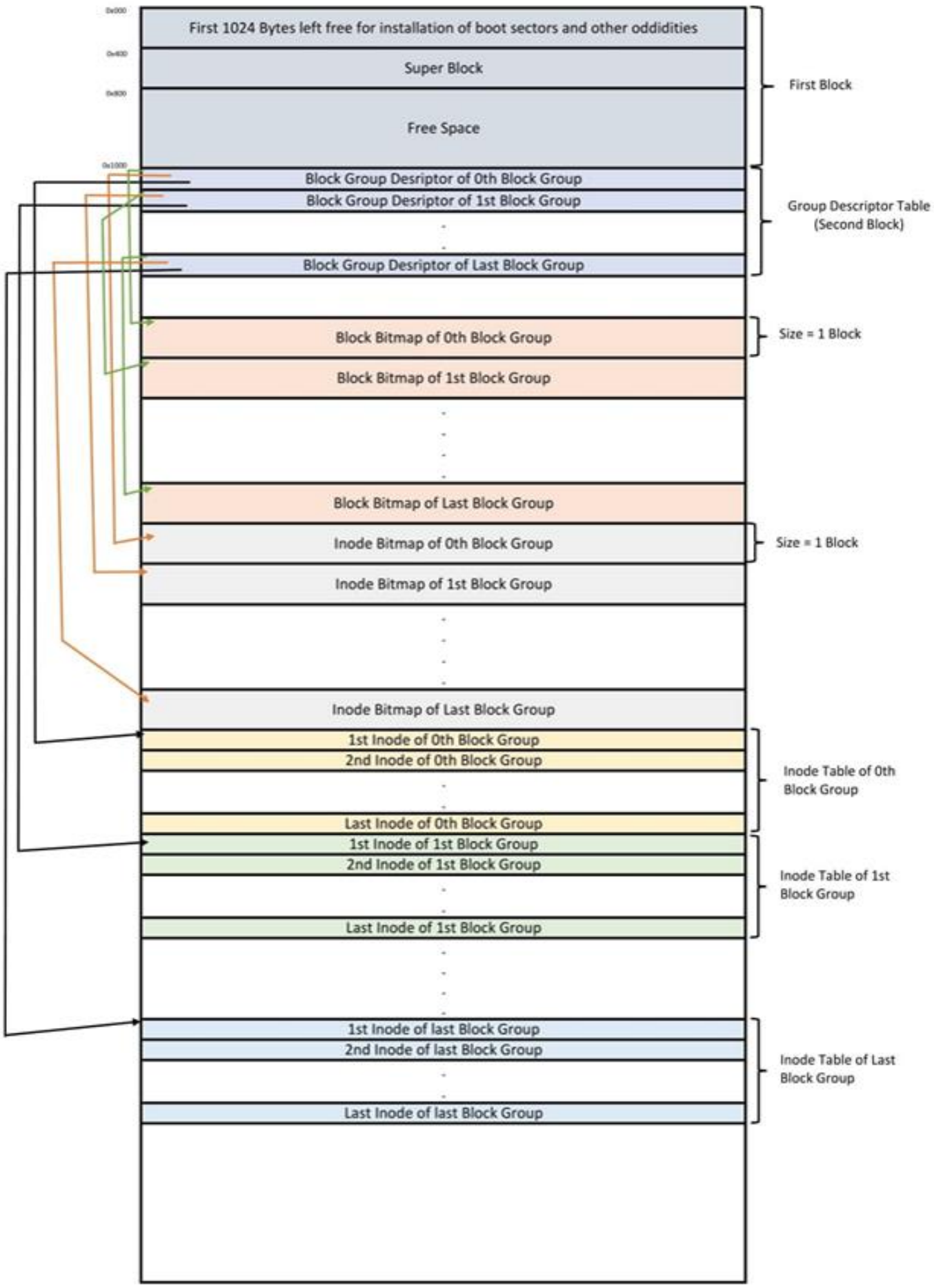
32-bit filesystem limits

Item	1KiB	2KiB	4KiB	64KiB
Blocks	2^32	2^32	2^32	2^32
Inodes	2^32	2^32	2^32	2^32
File System Size	4TiB	8TiB	16TiB	256TiB
Blocks Per Block Group	8,192	16,384	32,768	524,288
Inodes Per Block Group	8,192	16,384	32,768	524,288
Block Group Size	8MiB	32MiB	128MiB	32GiB
Blocks Per File, Extents	2^32	2^32	2^32	2^32
Blocks Per File, Block Maps	16,843,020	134,480,396	1,074,791,436	4,398,314,962,956 (really 2^32 due to field size limitations)
File Size, Extents	4TiB	8TiB	16TiB	256TiB
File Size, Block Maps	16GiB	256GiB	4TiB	256TiB

ext4 block group concept



Disc layout


0x000
First 1024 Bytes left free for installation of boot sectors and other oddidities
0x400
Super Block
0x800
Free Space
First Block
0x1000
Block Group Desriptor of 0th Block Group
Block Group Desriptor of 1st Block Group
Block Group Desriptor of Last Block Group
Group Descriptor Table (Second Block)
Block Bitmap of 0th Block Group
Block Bitmap of 1st Block Group
Block Bitmap of Last Block Group
Size = 1 Block
Inode Bitmap of 0th Block Group
Inode Bitmap of 1st Block Group
Inode Bitmap of Last Block Group
Size = 1 Block
1st Inode of 0th Block Group
2nd Inode of 0th Block Group
Last Inode of 0th Block Group
Inode Table of 0th Block Group
1st Inode of 1st Block Group
2nd Inode of 1st Block Group
Last Inode of 1st Block Group
Inode Table of 1st Block Group
1st Inode of last Block Group
2nd Inode of last Block Group
Last Inode of last Block Group
Inode Table of Last Block Group

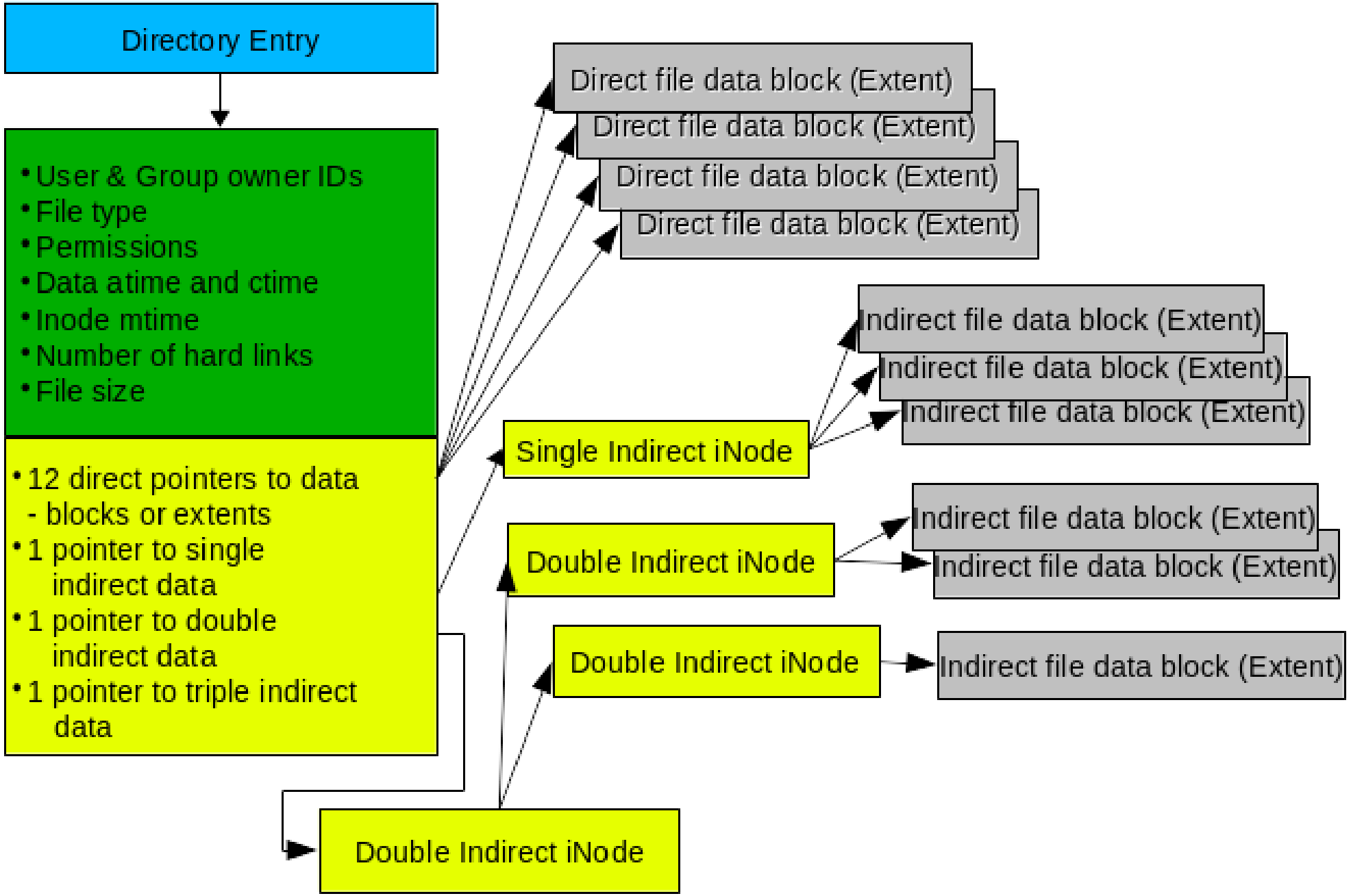
Byte level view of an Ext4 Superblock

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x000	s_inodes_count (Total inode count)				s_blocks_count_lo (Total block count)				s_r_blocks_count_lo				s_free_blocks_count_lo			
0x010	s_free_inodes_count				s_first_data_block				s_log_block_size				s_log_cluster_size			
0x020	s_blocks_per_group				s_clusters_per_group				s_inodes_per_group				s_mtime			
0x030	s_wtime				s_mnt_count		s_max_mnt_count		s_magic (0xEF53)		s_state		s_errors		s_minor_rev_level	
0x040	s_lastcheck				s_checkinterval				s_creator_os				s_rev_level			
0x050	s_def_resuid		s_def_resgid		s_first_ino (non reserved)				s_inode_size		s_block_group_nr		s_feature_compat			
0x060	s_feature_incompat				s_feature_rc_compat				s_uuid[16] (128-bit UUID)							
0x070	s_uuid[16] (128-bit UUID)								s_volume_name[16] Volume label							
0x080	s_volume_name[16] Volume label															
0x090	s_last_mounted[64] Directory where filesystem was last mounted															
0x0A0																
0x0B0																
0x0C0									s_algorithm_usage_bitmap				s_prealloc_blocks	s_prealloc_dir_blocks	s_reserved_gdt_blocks	
0x0D0	s_journal_uuid[16] UUID of journal super block															
0x0E0	s_journal_inum				s_journal_dev				s_last_orphan							
0x0F0	s_hash_seed[4] HTREE hash seed												s_def_hash_version	s_jnl_backup_type	s_desc_size	
0x100	s_default_mount_opts				s_first_meta_bg				s_mkfs_time							
0x110	s_jnl_blocks[17]															
0x120																
0x130																
0x140																
0x150	s_blocks_count_hi				s_r_blocks_count_hi				s_free_blocks_count_hi				s_min_extra_isize		s_want_extra_isize	
0x160	s_flags				s_raid_stride		s_mmp_interval		s_mmp_block							
0x170	s_raid_stripe_width				s_log_groups_per_flex	s_checksum_type	s_reserved_pad		s_kbyptes_written							
0x180	s_snapshot_inum				s_snapshot_id				s_snapshot_r_blocks_count							
0x190	s_snapshot_list				s_error_count (Number of errors seen)				s_first_error_time				s_first_error_ino			
0x1A0	s_first_error_block															
0x1B0	s_first_error_func[32]															
0x1C0									s_first_error_line				last_error_time			
0x1D0	s_last_error_ino				s_last_error_line				s_last_error_block							
0x1E0	s_last_error_func[32] (name of the function where the most recent error happened)															
0x1F0																

Layout of standard block group

Group 0 Padding	ext4 Super Block	Group Descriptors	Reserved GDT Blocks	Data Block Bitmap	inode Bitmap	inode Table	Data Blocks
1024 bytes	1 block	many blocks	many blocks	1 block	1 block	many blocks	many more blocks

inode stores information about each file



Special inodes

inode Number	Purpose
0	Doesn't exist; there is no inode 0.
1	List of defective blocks.
2	Root directory.
3	User quota.
4	Group quota.
5	Boot loader.
6	Undelete directory.
7	Reserved group descriptors inode. ("resize inode")
8	Journal inode.
9	The "exclude" inode, for snapshots(?)
10	Replica inode, used for some non-upstream feature?
11	Traditional first non-reserved inode. Usually this is the lost+found directory. See s_first_ino in the superblock.

some secret info

- An ext3 filesystem and an ext4 filesystem walk into a bar.
- The ext3 file system orders a drink and waits for it block by block.
- The ext4 file system just stands there. After ten minutes, the bartender asks, "Aren't you going to order anything?"
- The ext4 file system says, "I already did. I'm just using delayed allocation to see how thirsty I am before I commit to a glass.“

- NTFS doesn't get invited to Linux parties because it’s the guest that refuses to speak the language and leaves a mess behind.